SCHEDULING ALGORITHMS

FIRST COME FIRST SERVE(FCFS)

CODE:

```
#include <stdio.h>

int main(){
        int n;
        printf("Enter number of processes: ");
        scanf("%d",&n);

        int at[n], bt[n], ct[n], tat[n], wt[n];

        for(int i=0; i<n; i++){
                printf("Enter Arrival time and Burst Time for P%d: ", i+1);
                scanf("%d %d:",&at[i],&bt[i]);
        }

        float total_tat = 0, total_wt = 0;

        ct[0] = at[0] + bt[0];

        for(int i=0; i<n; i++){
                if(ct[i-1]<at[i]){
                        ct[i] = at[i] + bt[i];
                } else {
                        ct[i] = ct[i-1] + bt[i];
                }
        }

        for(int i=0; i<n; i++){
                tat[i] = ct[i] - at[i];
                wt[i] = tat[i] - bt[i];

                total_tat += tat[i];
                total_wt += wt[i];
        }

        printf("\nAverage Turnaround Time: %.2f", total_tat/n);
        printf("\nAverage Waiting Time: %.2f", total_wt/n);

        return 0;
}
```

OUTPUT:

```
[user123@localhost scheduling]$ gcc fcfs.c
[user123@localhost scheduling]$ ./a.out
Enter number of processes: 4
Enter Arrival time and Burst Time for P1: 0 2
Enter Arrival time and Burst Time for P2: 1 2
Enter Arrival time and Burst Time for P3: 5 3
Enter Arrival time and Burst Time for P4: 6 4

Average Turnaround Time: 3.50
Average Waiting Time: 0.75[user123@localhost scheduling]$ cat fcfs.c
```

SHORTEST JOB FIRST(SJF)

CODE:

```
#include <stdio.h>

int main(){
        int n;
        printf("Enter number of processes: ");
        scanf("%d",&n);

        int at[n], bt[n], ct[n], tat[n], wt[n];
        int done[n];

        for(int i=0; i<n; i++){
                printf("Enter Arrival Time and Burst Time for P%d: ",i+1);
                scanf("%d %d",&at[i], &bt[i]);
                done[i] = 0;
        }

        int completed = 0;
        int completionTime = 0;
        float avgtat = 0, avgwt = 0;

        while(completed < n){
                int idx = -1;
                int minBT = 9999;

                for(int i=0; i<n; i++){
                        if(at[i]<=completionTime && done[i] == 0){
                                if(bt[i]<minBT){
                                        minBT = bt[i];
                                        idx = i;
                                }
                        }
                }

                if(idx != -1){
                        if(completionTime<at[idx]){
                                completionTime = at[idx];
                        }
                        completionTime += bt[idx];
                        tat[idx] = completionTime - at[idx];
                        wt[idx] = tat[idx] - bt[idx];

                        done[idx] = 1;
                        completed++;
                } else {
                        completionTime++;
                }
        }

        for(int i=0; i<n; i++){
                avgtat += tat[i];
                avgwt += wt[i];
        }

        printf("\nAverage Turnaround Time: %.2f\n",avgtat/n);
        printf("Average Waiting Time: %.2f\n",avgwt/n);

        return 0;
}
[user123@localhost scheduling]$
```

OUTPUT:

```
[user123@localhost scheduling]$ gcc sjf.c
[user123@localhost scheduling]$ ./a.out
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 1 3
Enter Arrival Time and Burst Time for P2: 2 4
Enter Arrival Time and Burst Time for P3: 1 2
Enter Arrival Time and Burst Time for P4: 4 4

Average Turnaround Time: 6.25
Average Waiting Time: 3.00
[user123@localhost scheduling]$ cat sjf.c
```

ROUND ROBIN

CODE:

```
[user123@localhost ~]$ ls
a.out    Documents  fork.c      pc        sample.txt   Templates  Videos
banker   Downloads  Music       Pictures  scheduling   tharika
Desktop  exec.c     myfile.txt  Public    systemcalls  user123
[user123@localhost ~]$ mkdir scheduling
mkdir: cannot create directory ‘scheduling’: File exists
[user123@localhost ~]$ cd scheduling
[user123@localhost scheduling]$ vi roundrobin.c
[user123@localhost scheduling]$ cat roundrobin.c
#include <stdio.h>

int main()
{
    int n, tq, completed = 0, completionTime = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    printf("Enter Time Quantum: \n");
    scanf("%d", &tq);

    int at[n], bt[n], rem[n], tat[n], wt[n];
    float avgtat = 0, avgwt = 0;

    for (int i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: \n", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];
    }

    while (completed < n)
    {
        int idle = 1;

        for (int i = 0; i < n; i++)
        {
            if (at[i] <= completionTime && rem[i] > 0)
            {
                idle = 0;

                if (rem[i] > tq)
                {
                    completionTime += tq;
                    rem[i] -= tq;
                }
                else
```

```
                    completionTime += tq;
                    rem[i] -= tq;
                }
                else
                {
                    completionTime += rem[i];
                    tat[i] = completionTime - at[i];
                    wt[i] = tat[i] - bt[i];
                    rem[i] = 0;
                    completed++;
                }
            }
        }

        if (idle)
            completionTime++;
    }

    for (int i = 0; i < n; i++)
    {
        avgtat += tat[i];
        avgwt += wt[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", avgtat / n);
    printf("Average Waiting Time: %.2f\n", avgwt / n);

    return 0;
}
```

OUTPUT:

```
[user123@localhost scheduling]$ gcc roundrobin.c
[user123@localhost scheduling]$ ./a.out
Enter number of processes:
3
Enter Time Quantum:
2
Enter AT and BT for P1:
0 5
Enter AT and BT for P2:
1 3
Enter AT and BT for P3:
2
1

Average Turnaround Time: 6.33
Average Waiting Time: 3.33
[user123@localhost scheduling]$
```

PRIORITY

CODE:

```
[user123@localhost ~]$ ls
a.out    Documents  fork.c      pc        sample.txt   Templates  Videos
banker   Downloads  Music       Pictures  scheduling   tharika
Desktop  exec.c     myfile.txt  Public    systemcalls  user123
[user123@localhost ~]$ mkdir sched
[user123@localhost ~]$ cd sched
[user123@localhost sched]$ vi priority.c
[user123@localhost sched]$ cat priority.c
#include <stdio.h>

int main()
{
    int n, completed = 0, time = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int at[n], bt[n], priority[n], done[n];
    int ct[n], tat[n], wt[n];
    float avgtat = 0, avgwt = 0;

    for (int i = 0; i < n; i++)
    {
        printf("Enter AT, BT and priority for P%d: \n", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &priority[i]);
        done[i] = 0;
    }

    while (completed < n)
    {
        int idx = -1;
        int highPriority = 9999;

        for (int i = 0; i < n; i++)
        {
            if (at[i] <= time && done[i] == 0)
            {
                if (priority[i] < highPriority)
                {
                    highPriority = priority[i];
                    idx = i;
                }
            }
        }

        if (idx != -1)
        {
```

```
                    highPriority = priority[i];
                    idx = i;
                }
            }
        }

        if (idx != -1)
        {
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            done[idx] = 1;
            completed++;
        }
        else
        {
            time++;
        }
    }

    for (int i = 0; i < n; i++)
    {
        avgtat += tat[i];
        avgwt += wt[i];
    }

    printf("\nAverage Turnaround time: %.2f\n", avgtat / n);
    printf("Average Waiting Time: %.2f\n", avgwt / n);

    return 0;
}
```

OUTPUT:

```
[user123@localhost sched]$ gcc priority.c
[user123@localhost sched]$ ./a.out
Enter number of processes:
3
Enter AT, BT and priority for P1:
0 10 3
Enter AT, BT and priority for P2:
1 2 1
Enter AT, BT and priority for P3:
2 5 2

Average Turnaround time: 12.00
Average Waiting Time: 6.33
[user123@localhost sched]$
```